

Project Report: Placement Portal Application

Student Details

- **Full Name:** Jigyasa Doharey
- **Roll Number:** 24f2001460
- **Student Email:** 24f2001460@ds.study.iitm.ac.in
- **About Me:** An enthusiastic student pursuing Data Science and Applications at IIT Madras, specializing in modern web applications, Python backend development, and database systems.

Project Details

The goal of this project is to develop a secure, multi-role **Placement Portal Application** that bridges the gap between students, companies, and the institute placement admin. The system streamlines student registration and profile updates (including resume uploads), handles company approvals, processes job drive proposals, enforces branch/CGPA eligibility logic, conducts async exports (CSV), and manages automated background jobs (daily email/GChat alerts and monthly reports).

AI/LLM Declaration

- **Extent of Use:** I have used ChatGPT (GPT-5.5) to assist me in writing frontend template's bootstrap and the flow of vue part. The extent of AI/LLM usage is around 20-30% for frontend and also I also took help from it for doing the backend task around 15-20%. All AI-assisted designs, All final implementation logic, debugging, and integration were manually validated to ensure correctness and alignment with projects's constraints.

Technologies Used

- **Backend Framework:** **Flask** (Python) - Used for its lightweight, modular architecture to build robust JSON REST APIs.
- **Frontend Framework:** **Vue 3 + Vite** - Selected for modular, single-page application (SPA) rendering, responsive bindings, and fast client-side routing.
- **Database & ORM:** **SQLite & SQLAlchemy** - Provides a structured local SQL relational engine with a Python ORM mapping.
- **Authentication:** **Flask-JWT-Extended** - Facilitates role-based access control (RBAC) via secure JSON Web Tokens.
- **Caching Layer:** **Flask-Caching & Redis** (DB 1) - Caches heavy admin lists to optimize latency, automatically invalidated on state updates.
- **Async Job Worker:** **Celery & Redis** (DB 0) - Dispatches long-running export tasks and runs scheduled cron-like background jobs.
- **Styling System:** **Vanilla CSS** (Custom wireframe design system) + **Bootstrap 5** (Layout helpers) - Maximizes rendering layout responsiveness.

DB Schema Design (ER DIAGRAM)

The schema uses a relational database structured around a unified **User** model, subclassed/linked to role profiles to enforce data integrity:

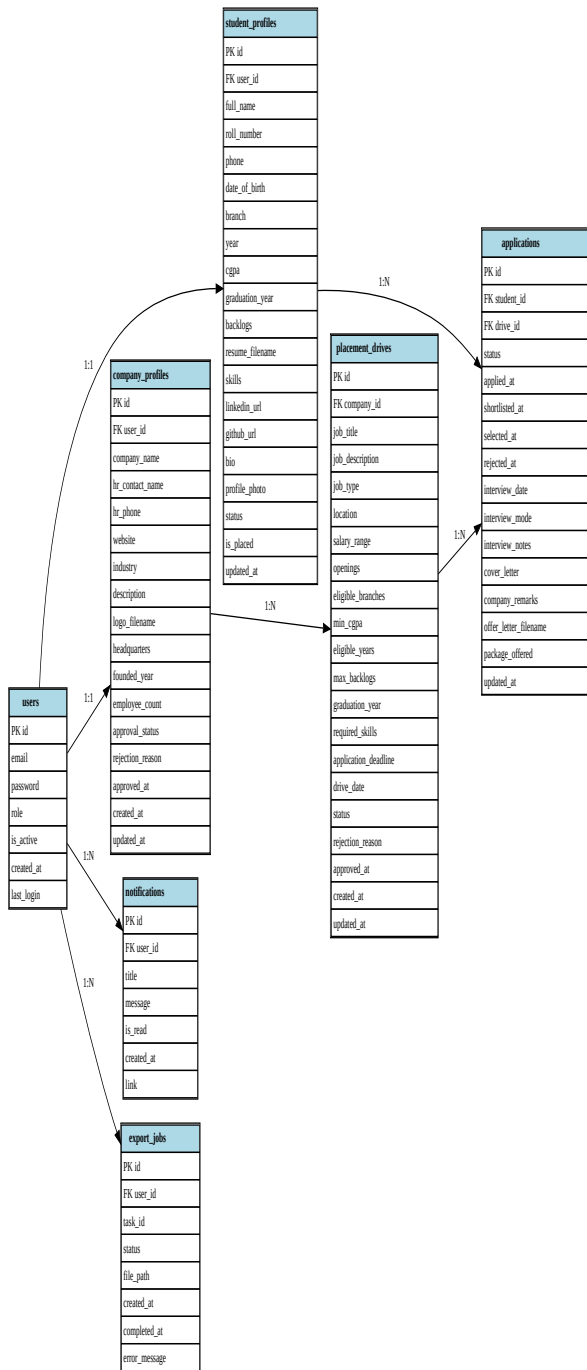


Table Details & Constraints

- users**: Centrally manages authentication. Enforces a **unique** constraint on the **email** column.
- student_profiles** & **company_profiles**: Extend the **users** table via foreign keys with **onDelete='CASCADE'**. Contain role-specific metadata.
- placement_drives**: Tied to **company_profiles.id**. Enforces eligibility parameters (**min_cgpa**, **max_backlogs**, **eligible_branches**).
- applications**: Enforces **unique_student_drive UniqueConstraint** on (**student_id**, **drive_id**) to prevent multiple duplicate applications to the same drive.

API Design

The application communicates via RESTful JSON endpoints grouped into modular Flask Blueprints:

- Authentication (/api/auth):**
 - POST /login** - Issues JWT token after checking password hash.
 - POST /register/student** - Self-registers a student account and profile.
 - POST /register/company** - Registers a company profile (defaults status to **pending**).
- Admin Dashboard (/api/admin):**
 - GET /dashboard** - Fetches total registration metrics and pending drive/company counts.
 - PUT /companies/<id>/approve** - Admin approves or rejects a company registration.

- `PUT /drives/<id>/approve` - Admin approves or rejects a placement drive proposal.
- `PUT /students/<id>/blacklist` - Deactivates student login and updates their profile status.
- **Company Dashboard (/api/company):**
- `POST /drives` - Proposes a new drive with eligibility constraints.
- `PUT /applications/<id>/status` - Shortlists, selects, or rejects an application.
- **Student Dashboard (/api/student):**
- `GET /drives?search=&eligible_only=` - Queries and filters approved drives.
- `POST /drives/<id>/apply` - Enforces eligibility checks, saves cover letter and submits.
- `POST /applications/export` - Triggers async Celery task to compile application history CSV.
- `GET /applications/export/download/<id>` - Retrieves the compiled CSV download.

Architecture and Features

Code Organization

- **backend/**: Organizes database models in `models/models.py`, API endpoints under `routes/` (blueprints: `admin.py`, `auth.py`, `company.py`, `student.py`), background Celery worker tasks in `tasks.py`, and app startup in `run.py`.
- **frontend/src/**: Houses components, router configuration in `router/index.js`, styling systems in `assets/wireframe.css`, and views in `views/` (`AdminDashboard.vue`, `CompanyDashboard.vue`, `StudentDashboard.vue`, etc.).

Key Features Implemented

- **Automatic Seed Mechanism**: Populates database with structured dummy data.
- **Eligibility Filter & Enforcer**: Prevents students from applying if branch, minimum CGPA, or backlog checks fail.
- **Dynamic Search**: Live search queries for companies, placement drives, and students.
- **Background Reminders (Scheduled)**: A Celery Beat scheduler executing daily (8:00 AM) to mail deadlines reminders and trigger Google Chat webhooks.
- **Monthly Admin Report (Scheduled)**: Celery task mailing a compiled HTML recruitment dashboard on the 1st of every month.

Presentation Video

- **Presentation Video Link**: [Google Drive Video Link - Presentation](#)